

第二章 文件、目录、文本处理与归档传输

文件题把路径、对象类型、内容筛选和传输连接在一起。结果文件存在并不代表找到的是正确对象，归档可打开也不代表成员路径符合要求。本章从目录树和链接语义出发，建立“选择对象 → 处理内容 → 归档或传输 → 验证成员与目标”的闭环。

[概念] 绝对路径从 `/` 开始，含义不依赖当前目录；相对路径从当前工作目录解释。inode 保存文件元数据并指向数据，目录项把名称映射到 inode；硬链接共享 inode，符号链接保存另一个路径文本。

[概念] 文件类型、文件名和内容是三个不同筛选维度。扩展名不决定 Linux 文件类型；`find` 依据目录树与元数据选择对象，`grep` 依据文本内容选择行。

[操作语义] `cp` 复制对象，`mv` 改名或移动目录项，`rm` 删除目录项，`install` 可在复制时设置目录、所有者和模式；`stat`、`file`、`du` 与 `df` 分别观察元数据、内容类型、目录占用与文件系统容量。

[操作语义] `tar` 把成员及元数据组织为归档；`scp` 通过 SSH 复制，`sftp` 提供交互传输，`rsync` 比较源和目标并同步差异。传输成功后仍要在目标端核对路径、大小和内容。

[知识专题] 路径、inode 与链接的判断边界

① [知识点] 目录的执行权限控制路径穿越

文件的 `r` 允许读取内容，`w` 允许修改内容，`x` 允许作为程序执行；目录的 `r` 允许列出名称，`w` 允许创建和删除目录项，`x` 允许穿越并访问已知名称。某个文件本身可读，但任一父目录缺少 `x`，访问仍会失败。

`pwd` 确认当前目录，`readlink -f` 解析可解析的最终路径，`namei -l` 逐层显示路径权限。排错时不要只检查最后一个文件的 `ls -l`。

② [知识点] 硬链接与符号链接失效方式不同

硬链接是同一文件 inode 的另一个名称，只能在同一文件系统内创建，通常不能指向目录。删除一个硬链接只减少链接计数，其他名称仍可访问数据。符号链接拥有自己的 inode，内容是目标路径，可跨文件系统并可指向目录；目标移动或删除后它会悬空。

```
ln /srv/report /srv/report.hard      # 同一 inode 的新名称
ln -s /srv/report /opt/report.current # 保存目标路径
stat -c '%i %h %n' /srv/report /srv/report.hard # inode 与链接数
readlink /opt/report.current          # 查看链接保存的路径
```

相对符号链接从链接所在目录解释，不是从创建命令时的工作目录解释。迁移一组目录时，相对链接可能更便携，但必须推演目标位置。

③ [验证点] 区分对象、目标与数据

用 `stat` 比较 inode 和链接计数，用 `test -L` 识别符号链接，用 `readlink -f` 查看最终目标。内容校验可用 `sha256sum`，但相同校验值只证明字节相同，不证明权限、所有者、时间戳或链接关系相同。

[Cheatsheet] 路径逐层权限：`namei -l`；元数据：`stat`；硬链接同 inode、不能跨文件系统；符号链接保存路径、可悬空；字节验证用 `sha256sum`。

[操作专题] 安全创建、复制、移动和删除对象

① [操作点] 创建目标前确认父目录和覆盖边界

`mkdir -p` 创建缺失父目录且对已存在目录不报错。`cp -a` 尽量保留递归结构和元数据，普通 `cp -r` 只表达递归；考试题是否需要保留所有者、权限和时间，应由目标终态决定。`install -D -m` 适合一次创建父目录、复制文件并设置模式。

```
mkdir -p /srv/project/archive          # 建立目录树
install -D -m 0640 source.conf /etc/app/app.conf # 复制并设模式
cp -a /srv/project/. /backup/project/    # 包含隐藏成员
```

源目录末尾的 `/.` 表示复制其内容；复制目录本身还是内容会改变目标层级。执行前用 `ls -ld` 和 `find <DIR> -maxdepth 1` 检查两端。

② [边界] 删除与移动作用于目录项

`rm -r` 和覆盖式 `mv` 可能不可恢复。通配符结果必须先用非破坏性命令预览，变量必须引用。不要用 `rm -rf` 作为权限或挂载排错工具；挂载点未挂载时删除目录内容，可能删除根文件系统中的数据。

③ [验证点] 元数据与内容分别验收

```
stat -c '%U %G %a %F %n' /etc/app/app.conf    # 所有者、组、模式、类型
sha256sum source.conf /etc/app/app.conf        # 字节内容
find /backup/project -maxdepth 2 -printf '%y %p\n' # 结构抽查
```

`du -sh` 统计目录实际占用，`df -hT <PATH>` 查询该路径所在文件系统的容量，两者对象不同，不能用 `du` 解释文件系统预留空间，也不能用 `df` 证明某目录内容大小。

[Cheatsheet] 建树：`mkdir -p`；复制并设模式：`install -D -m`；保留目录结构：`cp -a`；覆盖/删除前预览展开；验收：`stat + sha256sum + find`。

[操作专题] 用 find 选择对象，再批量处理

① [参数点] 起点、条件和动作必须分开读

`find <START> <TESTS> <ACTIONS>` 从起点递归。 `-type f` 选择普通文件， `-name` 区分大小写， `-iname` 忽略大小写， `-user / -group` 按身份， `-size` 按大小， `-mtime` 按 24 小时时段。

```
find /var/log -type f -name '*.log' -size +10M -print # 大于 10 MiB
find /home -user student -type f -mtime -7 -print    # 近 7 个 24h 周期
```

`+10M` 是严格大于， `-10M` 是严格小于， `10M` 是按 `find` 的单位向上取整后匹配。时间边界容易受取整影响，核心任务应结合 `stat` 抽查，不把一个条件机械解释为精确时间戳区间。

② [操作点] 批量动作避免空格和特殊字符破坏

优先使用 `-exec command {} +`，它把多个路径作为独立参数批量传递；需要逐文件动作时用 `\;`。向支持 NUL 的工具传递列表，可用 `-print0` 与 `xargs -0`。不要用 `for f in $(find ...)`，它会按空白分词。

```
find /srv/inbox -type f -name '*.txt' \
    -exec cp -t /srv/selected -- {} +          # 安全传递路径
```

复制前先把动作改为 `-print` 检查集合。 `--` 结束选项解析，避免以 `-` 开头的文件名被当成参数。

③ [验证点] 验证集合而不只看目标目录存在

保存源清单和目标清单，比较计数与代表性路径；需要字节一致时再计算校验值。目标目录中可能已有同名文件，单纯比较数量不能证明一一对应。

[Cheatsheet] `find` 起点 条件 动作；先 `-print` 预览；批量安全动作：`-exec ... {} +`；NUL 管道：`-print0 | xargs -0`；时间/大小边界用 `stat` 抽查。

[操作专题] 从文本中提取稳定证据

① [操作点] grep 选择行，字段工具再塑形

`grep -E` 使用扩展正则， `-i` 忽略大小写， `-n` 显示行号， `-v` 反选， `-r` 递归。模式以 `-` 开头时使用 `-e` 或 `--`。锚点 `^/$` 匹配行首/行尾，字符类 `[[:space:]]` 比在命令中输入不易辨认的空白更可靠。

```
grep -nE '^[[:space:]]*#[^[:space:]]' /etc/ssh/sshd_config # 非空非注释行
cut -d: -f1,3 /etc/passwd | sort -t: -k2,2n             # 用户名与 UID 排序
```

`sort` 排序, `uniq` 只折叠相邻重复行, 因此通常先排序; `wc -l` 统计换行记录; `tr` 做字符级替换。
`sed` 和 `awk` 在考试中应服务于明确的行或字段变换, 不写难以现场验证的长程序。

② [边界] 输出格式是接口

命令的面向人类默认输出可能随语言、终端宽度或版本变化。脚本或严格题目优先选择明确字段选项、稳定分隔符或专用查询格式。解析 `/etc/passwd` 可使用冒号字段, 但不要把带空格的普通输出硬拆成列。

③ [验证点] 对结果做规模、样本和反例检查

查看前后若干行、统计数量, 并用一个应匹配与一个不应匹配的样本验证模式。只有输出文件非空不足以证明正则没有过宽。

[Cheatsheet] 行筛选 `grep -nE`; 字段 `cut / awk`; 排序去重 `sort | uniq`; 计数 `wc -l`; 验证同时看数量、样本和反例。

[操作专题] 创建归档并通过 SSH 传输

① [参数点] 归档成员名称由工作目录和源路径决定

`tar -c` 创建, `-t` 列表, `-x` 提取, `-f` 指定归档; `-z` `gzip`, `-j` `bzip2`, `-J` `xz`。用 `-C` 先切换目录, 可避免归档中出现不需要的长前缀。

```
tar -C /srv -cJf /root/project.tar.xz project      # 成员以 project/ 开始
tar -tJf /root/project.tar.xz | sed -n '1,20p'    # 不解包检查成员
```

② [操作点] 根据传输语义选择工具

`scp source user@host:/path/` 适合直接复制; `sftp` 适合交互; `rsync -a` 递归并保留常用元数据, `-n` `dry-run`, `--delete` 会删除目标多余文件, 必须由题意明确授权。`rsync` 源目录末尾斜线表示复制目录内容, 不带斜线通常复制目录本身。

```
rsync -an /srv/project/ serverb:/srv/project/    # 先预览差异
rsync -a /srv/project/ serverb:/srv/project/     # 执行同步
```

③ [验证点] 本地归档与远端落点分别检查

先用 `tar -t` 检查成员与路径前缀, 再在目标端运行 `stat`、`sha256sum` 或第二次 `rsync -an`。SSH 命令退出 0 只说明传输工具没有报告失败, 不证明目标路径是题目要求的位置。

[Cheatsheet] 归档: `tar -C <BASE> -c[zjJ]f <ARCHIVE> <MEMBER>`; 查看: `tar -t...`; `rsync` 先 `-n`; 源末尾 `/` 决定复制内容还是目录; 目标端复核路径和校验值。

[诊断专题] 结果存在但层级、集合或内容不正确

① [诊断点] 多出一层目录

复制或 rsync 后目标出现 `/dest/source/source...`，先比较源末尾斜线和目标是否已存在，不要继续重复同步。用 `find -maxdepth 2` 看真实层级，再选择移动或重新以正确语义执行。

② [诊断点] find 漏掉含空格文件

若使用了命令替换或未引用变量，路径可能被拆分。回到 `-exec ... {} +` 或 NUL 分隔，不要通过禁止空格文件名掩盖管道错误。

③ [诊断点] 归档可打开但成员路径错误

`tar -t` 直接显示归档内名称。问题在创建时的工作目录和源参数，不应通过解包后手工移动来声称原归档合格；重新使用 `-C` 创建并验证。

[Cheatsheet] 层级错查源/目标末尾 `/`；集合漏项查分词与错误通道；归档路径错查 `tar -t` 和 `-C`；结果存在不等于对象关系正确。

[经典任务] 筛选日志、生成归档并传送到指定主机

从 `/var/log/app` 中找出所有者为 `appsvc`、大小超过 1 MiB、七个 24 小时周期内修改的普通 `.log` 文件。路径可能包含空格。将文件复制到 `/srv/submission/logs`，保留其相对于起点的目录结构；生成 `/srv/submission/app-logs.tar.xz`，归档成员必须从 `logs/` 开始；传送到 `serverb:/srv/archive/`。

不得删除或改写源文件，不得把查找错误当作成员路径。验收时证明筛选集合、复制层级、归档成员、远端落点和归档校验值。

请先独立完成。参考解答从下一页开始。

[参考解答] 用 NUL 路径流保留层级并分层验收

① [操作点] 预览集合与边界

```
find /var/log/app -type f -user appsvc -name '*.log' \
    -size +1M -mtime -7 -print # 先人工抽查
```

确认条件后创建目标。GNU `cp --parents` 能保留给定路径结构；先切到起点，避免把 `/var/log/app` 整段带入目标。

② [操作点] 安全复制路径集合

```
mkdir -p /srv/submission/logs
cd /var/log/app
find . -type f -user appsvc -name '*.log' -size +1M -mtime -7 \
    -exec cp --parents -- {} /srv/submission/logs/ \; # 保留相对层级
```

③ [操作点] 创建并传输归档

```
tar -C /srv/submission -cJf /srv/submission/app-logs.tar.xz logs
scp /srv/submission/app-logs.tar.xz serverb:/srv/archive/
```

④ [验证点] 五层验收

```
find /srv/submission/logs -type f -printf '%P\n' | sort
tar -tJf /srv/submission/app-logs.tar.xz | sed -n '1,20p'
sha256sum /srv/submission/app-logs.tar.xz
ssh serverb 'stat /srv/archive/app-logs.tar.xz; sha256sum /srv/archive/app-logs.tar.xz'
```

比较源集合与目标相对路径，确认所有归档成员以 `logs/` 开始，并让本地与远端校验值一致。若 `find` 有权限错误，应单独保留并处理，不能静默当作完整集合。

[Cheatsheet] 预览条件 → 从起点生成相对路径 → `cp --parents` 保层级 → `tar -C` 控制成员前缀 → `scp` → 路径集合 + `tar -t` + 双端校验值。

[本章收束] 先证明选中了谁，再证明对它做了什么

文件操作的稳定顺序是先确认路径与对象类型，再建立筛选集合，最后执行复制、归档或传输。链接关系由 `inode` 与目标路径证明，文本筛选要检查规模、样本与反例，批量路径必须避免空白分词。

归档和远端文件都是新对象：归档成员名称、压缩格式、远端落点和字节内容需要分别验证。只要把“对象集合”和“处理结果”分成两层，文件题就不会因为一个存在的输出文件而提前结束。